Rest API

MERN STACK : Content

- Introduction to Rest API's
- HTTP verbs
- Questionnaire

Introduction to Rest - API's

- A REST API (Representational State Transfer - Application Programming Interface) is a set of rules that developers follow when they create APIs.
- APIs allow two software applications to communicate with each other over the web.

Key Concepts of REST API:

- **Resources:** In REST, everything is treated as a resource, which could be any object, data, or service. Resources are identified by URLs (Uniform Resource Locators). For example, a user's profile or a blog post can be a resource.

Introduction to Rest - API's

- **HTTP Methods:** REST APIs use standard HTTP methods to perform actions on resources. These are:
  - GET: Retrieve data from a resource.
  - POST: Create a new resource.
  - PUT: Update an existing resource.
  - DELETE: Remove a resource.
- **Statelessness:** Each request from the client to the server must contain all the information the server needs to fulfill the request. The server does not store any state between requests, making each API call independent of others.

Introduction to Rest - API's

- **CRUD Operations:** REST APIs often follow the CRUD model (Create, Read, Update, Delete) to manage resources:
  - Create: Corresponds to POST
  - Read: Corresponds to GET
  - Update: Corresponds to PUT or PATCH
  - Delete: Corresponds to DELETE
- **JSON/XML Data Format:** REST APIs typically use JSON (JavaScript Object Notation) as the format for sending and receiving data because it's lightweight and easy to read. Some APIs might also support XML.

Introduction to Rest - API's

Example of a REST API Call: Let's say you have a REST API to manage blog posts. An example interaction could be:

- To retrieve all posts: GET /posts
- To retrieve a specific post: GET /posts/postid
- To create a new post: POST /posts with post details in the request body.
- To update a post: PUT /posts/postid with updated post details.
- To delete a post: DELETE /posts/postid

Introduction to Rest - API's

Benefits of REST APIs: Here are some of the benefits of REST APIs

- Scalability: Since REST APIs are stateless, they can handle large numbers of requests more easily, making them scalable.
- Flexibility: REST APIs can handle different types of calls, return different data formats, and even change structure with new versions.
- Interoperability: REST is language-agnostic, allowing it to be used by different systems and platforms.

HTTP verbs

- HTTP verbs (also known as HTTP methods) are fundamental building blocks of REST APIs.
- They define the type of action the client wants to perform on a resource, typically related to managing data (create, read, update, delete).
- Each verb corresponds to a different action.

HTTP verbs(Continued...)

Here's a breakdown of the most commonly used HTTP verbs:

- GET: Retrieve data from a server. This method is idempotent, meaning multiple identical requests will yield the same result.
- POST: Send data to the server to create a new resource. This method is not idempotent, as repeated requests can create multiple resources.
- PUT: Update an existing resource or create it if it does not exist. This method is idempotent.
- PATCH: Partially update an existing resource. Unlike PUT, which replaces the entire resource, PATCH modifies only the specified fields.
- DELETE: Remove a resource from the server. This method is also idempotent; deleting the same resource multiple times has the same effect as deleting it once.
- HEAD: Similar to GET but only retrieves the headers, not the body. Useful for checking resource availability without transferring data.

HTTP verbs(Continued...)

- OPTIONS: Describe the communication options for the target resource, allowing clients to understand what methods are supported.
- TRACE: Echo back the received request, used mainly for diagnostic purposes.
- CONNECT: Establish a tunnel to the server identified by the target resource, often used with HTTPS.

Questionnaire

Questions

Thank you

Web Servers

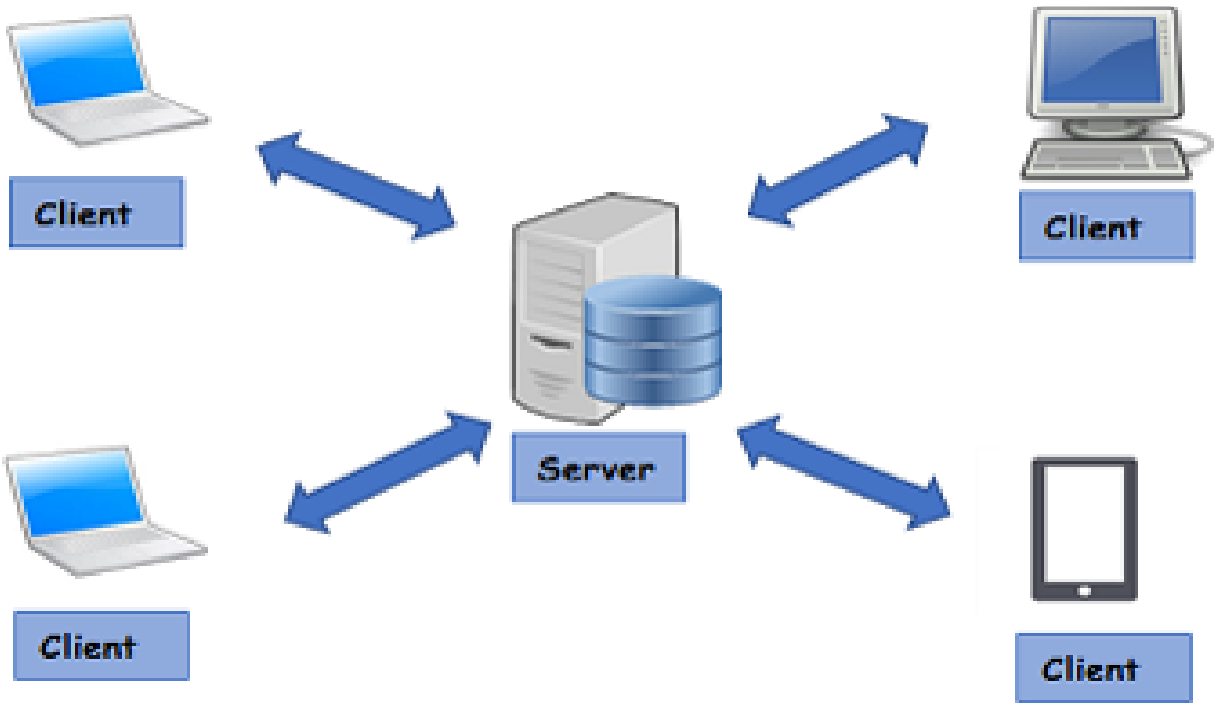
MERN STACK : Content

- Client-Server Architecture
- Request-Response objects
- Creating a Basic Http Server
- Questionnaire

Client - Server Architecture

- The client-server architecture refers to a system that hosts, delivers, and manages most of the resources and services the client requests.
- It is arranged that clients are often situated at workstations or on personal computers, while servers are located elsewhere on the network, usually on more powerful machines.

Client - Server Architecture:



Client - Server Architecture

- **Client:** The client is a device or application that requests services or resources from a server. It could be a web browser, mobile app, or any other software that interacts with the server.
- **Server:** The server is a powerful computer or software that provides services, processes requests, and manages resources. It could be a web server, database server, or application server.

How does Client - Server Model Work:

- Request-Response Model:
  - The client sends a request to the server, usually over a network (like the internet).
  - The server processes this request and returns a response to the client. The response could be a webpage, data, or a status message.

Client - Server Architecture

- Communication Protocols:
  - Clients and servers communicate using standardized protocols, such as HTTP/HTTPS for web services, FTP for file transfers, and SMTP for email.
- Stateless vs. Stateful:
  - **Stateless:** Each client-server interaction is independent. The server does not retain any memory of previous requests. HTTP is an example of a stateless protocol.
  - **Stateful:** The server retains information about the client across multiple requests. For example, a database server that maintains a session with a client is stateful.

Client - Server Architecture

Types of Client - Server Architecture:

1. **Two-Tier Architecture:**
   - The simplest form, where the client directly communicates with the server.
   - Example: A desktop application directly connecting to a database server.
2. **Three-Tier Architecture:**
   - Adds an intermediate layer, typically an application server, between the client and the database server.
   - Example: A web browser (client) interacts with a web server (application server), which then communicates with a database server.
3. **N-Tier Architecture:**
   - Extends the three-tier model by adding more layers, like load balancers, microservices, or caching systems.
   - Used in complex, large-scale applications to improve performance, scalability, and maintainability.

Client - Server Architecture

Advantages:

- **Centralized Control:** Servers manage resources centrally, making it easier to maintain, update, and secure the system.
- **Scalability:** Servers can be upgraded to handle more clients, and the system can be distributed across multiple servers.
- **Security:** Servers can enforce security policies, manage user authentication, and control access to resources.
- **Flexibility:** Clients can run on different platforms (e.g., web browsers, mobile devices) and communicate with the same server.

Examples:

- **Web Applications:** Browser (client) requests web pages from a web server.
- **Email Services:** Email clients like Outlook or Gmail request and send emails via email servers.
- **Online Games:** The game client connects to a central game server to sync gameplay, scores, and interactions.

Request - Response Objects

In the context of web servers, request and response objects are central to the communication between the client (e.g., a web browser) and the server.

Request Object:

`The request object represents the HTTP request made by the client to the server. It contains all the data that the client has sent to the server.`

Key Components of Request Object:

- **Request Method:** Specifies the action the client wants the server to perform. Common HTTP methods include:
  - GET: Requests data from the server (e.g., fetching a web page).
  - POST: Submits data to the server (e.g., form submission).
  - PUT: Updates existing data on the server.
  - DELETE: Deletes data from the server.

Request - Response Objects

- **Request URL:** The URL the client is requesting, which may include a path and query string.
  Example: /products?id=123 where /products is the path and id=123 is the query string.
- **Headers:**Key-value pairs that provide additional information about the request, such as:
  - Host: The domain name of the server.
  - User-Agent: Information about the client software (e.g., browser type).
  - Content-Type: he media type of the body of the request (e.g., application/json).
- **Body:**The data sent by the client in a POST, PUT, or PATCH request.
  Example: In a form submission, the body might contain the form data as key-value pairs.

Request - Response Objects

- **Cookies:** Small pieces of data sent by the client that the server can use to maintain state between requests (e.g., session identifiers).
- **Query Parameters:**Data included in the URL after the ? symbol, often used to pass information to the server.
  Example: /search?query=books.
- **Route-Paramters:**Dynamic parts of the URL that are used to capture values from the URL.
  Example: /users/:id where :id could be a specific user ID

Request - Response Objects

Response Object:

The response object represents the HTTP response that the server sends back to the client. It allows the server to send data, set headers, and control the status of the response.

Key Components of Response Object:

- **Status Code:** A numerical code that indicates the result of the request.

  Common codes include:
  - 200 OK: The request was successful.
  - 404 Not Found: The requested resource could not be found.
  - 500 Internal Server Error: The server encountered an error.
- **Headers:**Key-value pairs that provide additional information about the response, such as:
  - Content-Type: The media type of the response (e.g., text/html, application/json).
  - Set-Cookie: Sets a cookie on the client.

Request - Response Objects

- **Body:**The data sent back to the client. This could be HTML, JSON, XML, or any other type of data.
  Example: A JSON object containing user information.
- **Cookies:** The server can send cookies to the client, which the client will store and send with future requests.
- **Redirection:** The server can instruct the client to redirect to a different URL using a '3xx' status code and a 'Location'. header.

How Request and Response Work Together:

- Server Sends a Response: The server generates a response object, sets the appropriate status code, headers, and body content, and sends it back to the client.
- Client Receives the Response: The client receives and interprets the response, which could involve rendering a web page, displaying a message, or storing data.

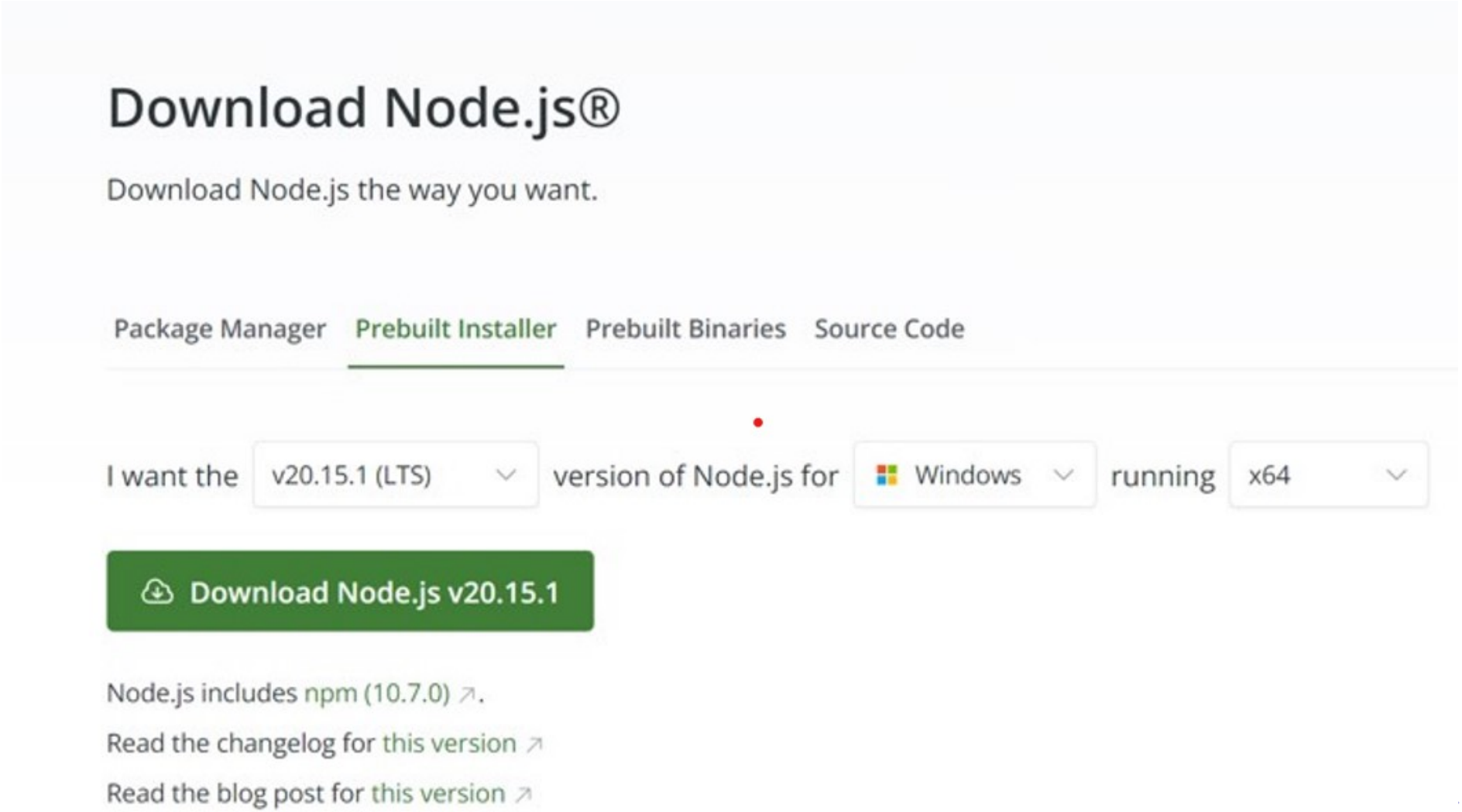
Creating a Basic Http Server

HTTP Server: Creating a basic HTTP server can be done in various programming languages.We are creating a simple HTTP server in Node.js using the built-in http module.

Note: We need to install Node.js to use built-in http module to create a simple Http server

Creating a Basic Http Server

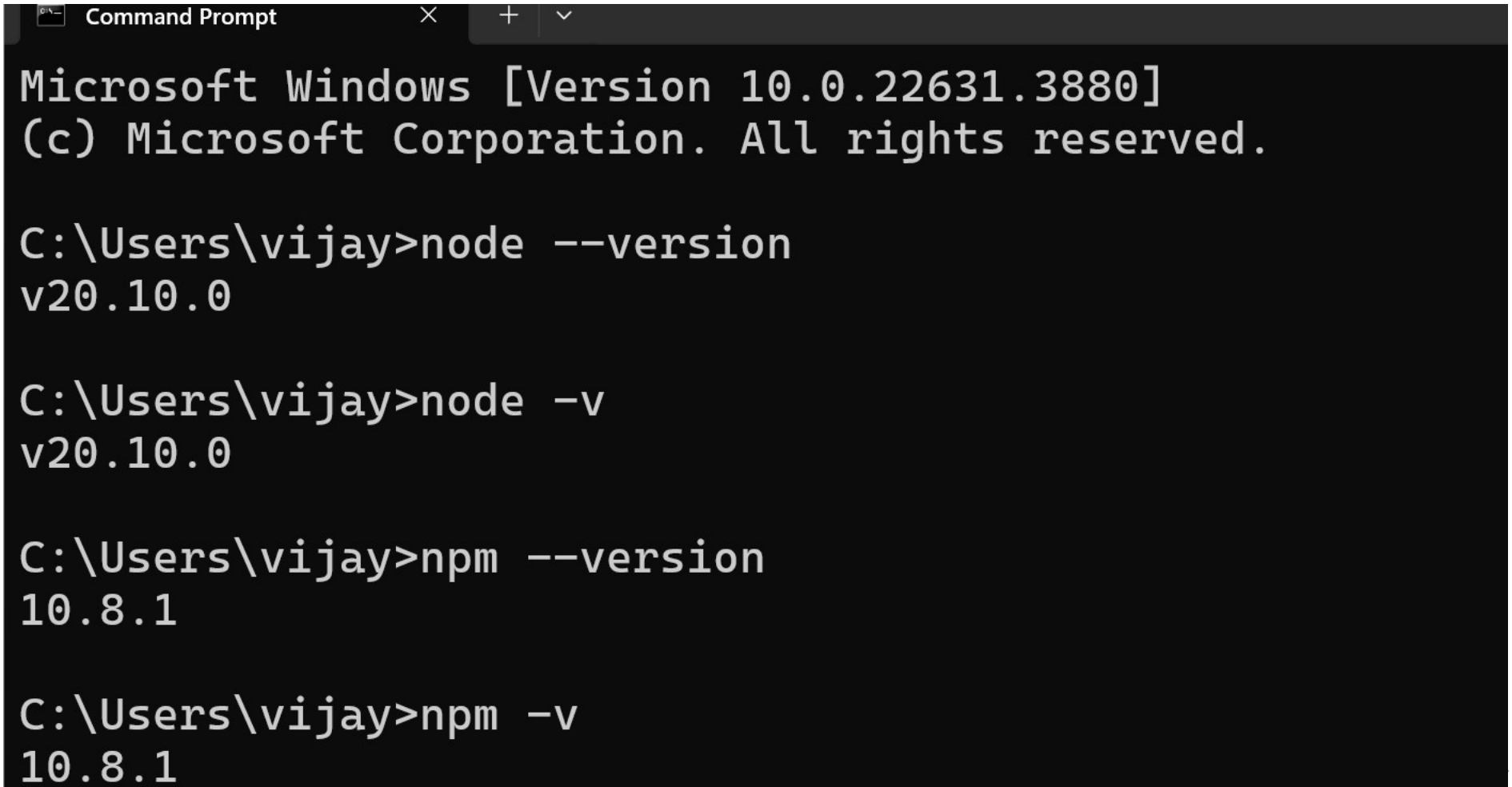
Install the NodeJs using the following Link:
https://nodejs.org/en/download/prebuilt-installer



Creating a Basic Http Server

We need to verify the NodeJs installation as below:

- Node: Go to the command prompt and type node –v or node –version
- npm: Go to the command prompt and type npm –v or npm –version



Creating a Basic Http Server

Step By Step Process to create a Http Server:

- First, ensure you have Node.js installed on your machine.
- Create a new directory for your project
- Open the new directory using VS code editor.
- Open Terminal and type the following command: npm init -y
  Note: This will create a package.json file
- Create a new file called server.js and write the following code:

```
// Import the built-in http module
const http = require('http');

// Define the hostname and port the server will listen on
const hostname = '127.0.0.1';
const port = 3000;

// Create an HTTP server
const server = http.createServer((req, res) => {
    res.statusCode = 200;
    res.setHeader('Content-Type', 'text/plain');
```

Creating a Basic Http Server

```
    // Write the response body
    res.end('Hello, World!\n');
});

// Start the server and listen on the specified hostname and
    port
server.listen(port, hostname, () => {
    console.log(`Server running at http://${hostname}:${port
    }/`);
});
```

Run: Run the application by typing node server.js at the terminal

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Admin\Music\WT\WT Samples\HTTP-Server> node server.js
Server running at http://127.0.0.1:3000/
```

Questionnaire

Questions

Thank you